Date :

13 · Java String

13.1 Introduction

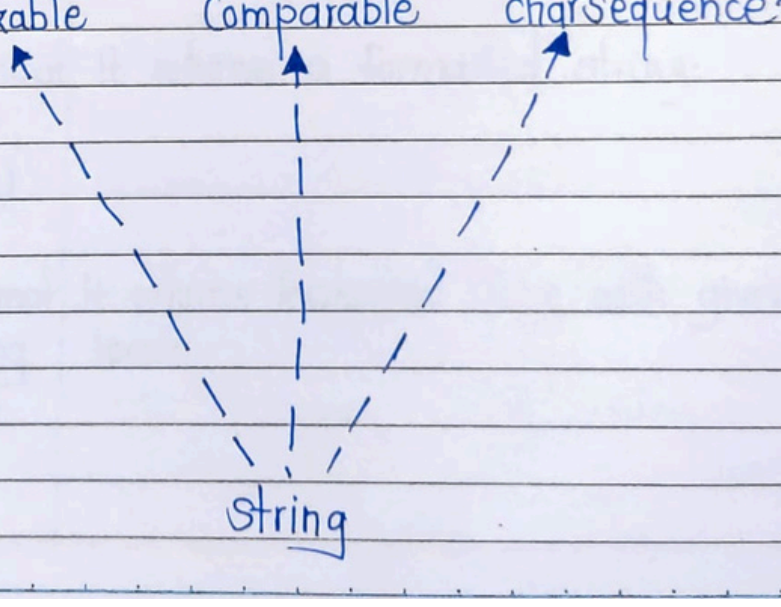
In Java, string is basically an object that represents sequence of char values. An array of characters works same as Java string. for example:

```
char[] ch = {'j', 'a', 'v', 'a', 'l', 'a', 'n', 'g', 'u', 'a', 'g', 'e'};
String s = new String(ch);
```

is same as :

```
String s = 'Javalanguage';
```

Java String class provides a lot of methods to perform operations on strings such as compare(), concat(), equals(), split(), length(), replace(), compareTo(), intern(), substring() etc.
The java.lang.string class implements Serializable, Comparable, charSequence interfaces.



Date : _____

String is a sequence of characters. But in Java, string is an object that represents a sequence of characters. The java.lang.String class is used to create a string object.

There are two ways to create string object:

1. By string literal
2. By new keyword

13.2 Java String class methods

The java.lang.String class provides many useful methods to perform operations on sequence of char values.

Method	Description
1) char charAt (int index)	It returns char value for the particular index
2) int length()	It returns string length.
3) static string format (string format, object... args)	It returns a formatted string.
4) static string format (Locale l, string format, object.. args)	It returns formatted string with given locale.

Date : _____

Method	Description
5) String substring (int beginIndex)	It returns substring for given begin index.
6) String substring (int beginIndex, int endIndex)	It returns substring for given begin index and end index.
7) boolean contains (charSequence s)	It returns true or false after matching the sequence of char value.
8) static String join (charSequence delimiter, char-Sequence .... elements)	It returns a joined string.
9) String intern()	It returns an interned string.
10) int indexOF (int ch)	It returns the specified char value index.
11) StringtoLowerCase()	It returns a string in lowercase.
12) StringtoUpperCase()	It returns a string in uppercase.
13) String trim()	It removes beginning and ending spaces of this string.

Date : _____

13.3 Java StringBuffer class

Java stringBuffer class is used to create mutable (modifiable) string objects. The StringBuffer class in Java is the same as string class except it is mutable. i.e it can be changed. Java stringBuffer class is thread-safe. i.e multiple threads cannot access it simultaneously. So it is safe and will result in an order.

Constructor	Description
stringBuffer()	It creates an empty string buffer with the initial capacity of 16.
stringBuffer(string str)	It creates a string buffer with the specified string.
stringBuffer(int capacity)	It creates an empty string buffer with the specified capacity as length.

13.4 Java stringBuilder class:

Java StringBuider class is used to create mutable (modifiable) string objects. The Java stringBuilder class is same as stringBuffer class except that it is non-synchronized. It is availdble since JDK 1.5

1) StringBuilder append() method

Date :

2) StringBuilder insert() method
3) StringBuilder replace() method
4) StringBuilder delete() method
5) StringBuilder reverse() method
6) StringBuilder capacity() method
7) StringBuilder ensureCapacity() method

Example:

```
class StringBuilderExample{
  public static void main(String args[]){
   StringBuilder sb = new StringBuilder("Hello");
   sb.append("Java");
   System.out.println(sb);
}
}
```

13.5 Java toString():

If you want to represent any object as a string, toString() method comes into existence. The toString() method returns the string representation of the object. If you print any object, java compiler internally invokes the toString() method on the object. So overriding the toString() method, returns the desired output, it can be the state of an object etc. depending on your implementation.

Advantages of Java toString() method:

By overriding the toString() method of the object class, we can return values of the object, so we don't need

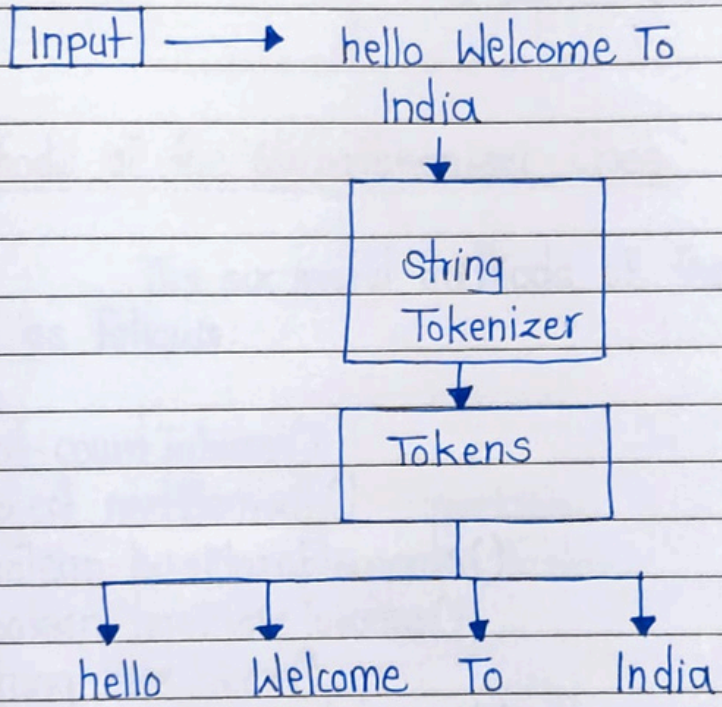
Date :

to write much code.

13.6 String Tokenizer in Java:

The java.util.StringTokenizer class allows you to break a string into tokens. It is simple way to break a string. It is a legacy class of Java. It doesn't provide the facility to differentiate numbers, quoted strings, identifiers etc. like StreamTokenizer class. We In the StringTokenizer class, the delimiters can be provided at the time of creation or more one by one to the tokens.

Example of String Tokenizer class in Java



Constructors of the StringTokenizer class:

There are 3 constructors defined in the String-Tokenizer class.

Date :

Constructor	Description
StringTokenizer(string str)	It creates stringTokenizer with specified string.
stringTokenizer(string str, string delim)	It creates stringTokenizer with specified string and delimiter.
StringTokenizer(string str, string delim, boolean returnvalue)	It creates stringTokenizer with specified string, delimiter and returnvalue. If return value is true, delimiter characters are considered to be tokens. If it is false, delimiter characters serve to seprate tokens.

Methods of the StringTokenizer Class :

The six useful methods of the StringTokenizer class are as follows:

1) int countTokens()
2) Object nextElement()
3) boolean hasMoreElements()
4) boolean hasMoreTokens()
5) String nextToken()
6) String nextToken(string delim)

Example of StringTokenizer class :

```
import java.util.StringTokenizer;
```

Date : _____

```
public class Simple{
  public static void main(String args[]){
    StringTokenizer st = new StringTokenizer("my name is
    yadnyesh"," ");
    while (st.hasMoreTokens()){
        System.out.println(st.nextToken());
    }
  }
}
```

Output:

```
my
name
is
yadnyesh
```